CSC2042S Assignment 3 ~ Adam Foflonker

Preamble

3rd Party Libraries Used

- Pandas
- NumPy
- Scikit-Learn
- PyTorch
- Matplotlib
- Seaborn

Running the Notebook

- Please run the *setup script* `setup.py`. This script will download the dataset to the required path and install all required dependencies to run the notebook.
- The dependency installation step can be skipped if you already have them installed.

Global Constants, State and Flags

A constant *random state* was defined to ensure code reproducibility across different machines. The following hyperparameters were defined as well:

```
MAX_EPOCHS: int = 50
MAX_FEATURES: int = 10000
MIN_DF: int = 2

RANDOM_SEED: int = 42   # Fixed random seeds for deterministic outputs
random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)
torch.manual_seed(RANDOM_SEED)
torch.cuda.manual_seed_all(RANDOM_SEED)
```

Tasks

Data Preprocessing

The *Masakhane* datasets for **English** and **isiXhosa** were loaded as two separate datasets - each containing their own separate **train**, **validation** and **test** sets along with their labels.

Basic cleaning was applied to each dataset as well - converting everything to lowercase, coalescing excessive whitespaces, and replacing *newlines* `\n` with whitespace using **regular expressions**.

Preprocessing also combines the `headline` and `text` columns into a **single feature**.

Multinomial Logistic Regression Implementation

A `MultinomialLogisticRegression` class was implemented which inherits from PyTorch's `nn.Module` class. This class has the following methods:

- `__init__(self, int, int)`: Constructs a new instance of this model.
- `forward(self, torch.Tensor)`: Returns logits for input batch x (shape: batch x input_dim).
- `compute_probabilities(self, torch.Tensor)`: Returns softmax probabilities for input batch.
- `predict(self, torch.Tensor)`: Returns the predicted class indices.

Training

The main training loop features `nn.CrossEntropyLoss` for multi-class classification. It also uses `torch.optim.Adam` as its *optimizer* as `Adam` often converges faster and is less sensitive to the initial learning rate than `SGD`.

An **epoch** count of `50` epochs was used as the stopping criterion as this yielded a higher accuracy than using *early stopping* methods.

Hyperparameter Tuning

For hyperparameter tuning, the *grid search* method was used to compare the performances of the **English** and **isiXhosa** models when being trained with different combinations of the **learning rate** and **batch size** hyperparameters. All these tests were done using the fixed `MAX_FEATURES` of `10000`. The hyperparameter value combination which yielded the highest accuracy (best hyperparameters) were then applied to the final models for **English** and **isiXhosa**.

Truncated grid search output:

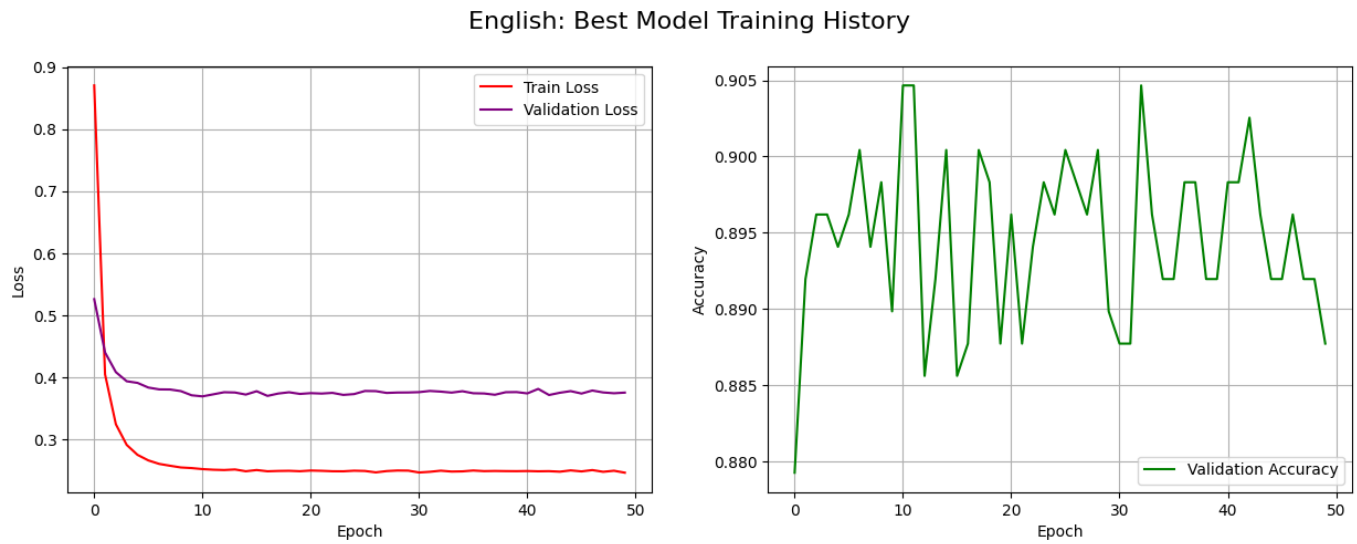
```
--- Starting Grid Search for ENGLISH ---
  Testing lr = 0.01, batch_size = 8...
  Testing lr = 0.01, batch_size = 16...
  Testing lr = 0.01, batch_size = 32...
...
--- Grid Search for ENGLISH Complete ---
Best validation accuracy: 0.9047
Best parameters: {'lr': 0.01, 'batch_size': 8}

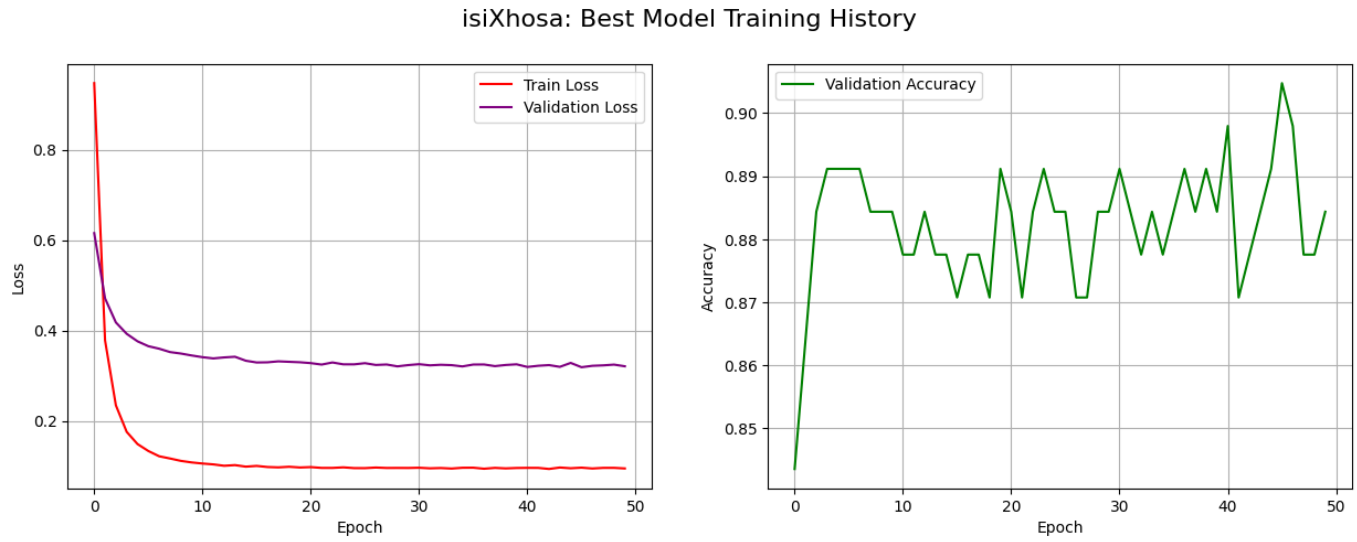
English Grid Search Results:
        lr  batch_size   val_acc
0   0.0100           8  0.904661
1   0.0100          16  0.904661
2   0.0100          32  0.902542
...
--- Starting Grid Search for ISIXHOSA ---
  Testing lr = 0.01, batch_size = 8...
  Testing lr = 0.01, batch_size = 16...
  Testing lr = 0.01, batch_size = 32...
...
--- Grid Search for ISIXHOSA Complete ---
Best validation accuracy: 0.9048
```

```
Best parameters: {'lr': 0.01, 'batch_size': 8}

isiXhosa Grid Search Results:
        lr  batch_size   val_acc
0   0.0100           8  0.904762
1   0.0100          16  0.897959
2   0.0100          32  0.891156
...
```

Training history curves:





It can be determined from the above results that the **best hyperparameters** for both models are:

- **Learning rate**: `0.01`
- **Batch size**: `8`

Feature Extraction

Three different methods for converting text into numerical vectors were compared, namely:

- **Bag-of-Words**
- **Binary Presence**
- **TF-IDF**

The optimal hyperparameters found previously were used for these feature extraction comparisons, and these comparisons were performed separately for the **English** and **isiXhosa** models.

Truncated feature extraction comparison output:

```
--- Starting Feature Extraction Experiments for ENGLISH ---
  Testing method = count, max_features = 2000...
  ...
  Testing method = binary, max_features = 2000...
  ...
  Testing method = tfidf, max_features = 2000...
  ...
--- Feature Experiments for ENGLISH Complete ---
Results:
   method  max_features   val_acc
0   count          2000  0.870763
...
3  binary          2000  0.847458
...
6   tfidf          2000  0.896186

Best feature config: method = 'tfidf', max_features = 10000 with accuracy 0.9047

--- Starting Feature Extraction Experiments for ISIXHOSA ---
  Testing method = count, max_features = 2000...
  ...
  Testing method = binary, max_features = 2000...
  ...
  Testing method = tfidf, max_features = 2000...
  ...
--- Feature Experiments for ISIXHOSA Complete ---
Results:
   method  max_features   val_acc
0   count          2000  0.904762
...
3  binary          2000  0.897959
...
6   tfidf          2000  0.925170
...

Best feature config: method = 'binary', max_features = 5000 with accuracy 0.9388
```

From this output we can see that the best feature extraction method for **English** is '**tfidf**' and for **isiXhosa** is '**binary**'.

isiXhosa Training Decisions

The isiXhosa dataset is smaller and more imbalanced than the English one, making it more prone to overfitting and biased towards majority classes. Two approaches to mitigate this have been investigated:

1. **Regularization**: Add a penalty term to the loss function to discourage large model weights, which helps prevent overfitting.
    - **L2 Regularization (Weight Decay)**: Penalizes the squared magnitude of the weights.
    - **L1 Regularization**: Penalizes the absolute value of the weights. It can lead to sparse weights (some become exactly zero), effectively performing feature selection.
2. **Handling Class Imbalance**: Rebalance the training data to ensure the model doesn't ignore minority classes.
    - **Upsampling Minority Classes**: Randomly duplicates samples from the minority classes until they have as many samples as the majority class.
    - **Downsampling Majority Classes**: Randomly removes samples from the majority classes until they have as many samples as the minority class.

The best hyperparameters found previously have been reused here as well.

isiXhosa special experiments output:

```
--- Starting isiXhosa Special Experiments ---
Original class distribution: {1: 350, 4: 347, 3: 215, 2: 70, 0: 50}
Upsampled class distribution: {0: 350, 1: 350, 2: 350, 3: 350, 4: 350}
Downsampled class distribution: {0: 50, 1: 50, 2: 50, 3: 50, 4: 50}

--- isiXhosa Special Experiments Complete ---
      technique    param   val_acc
0      Baseline      N/A  0.897959
1            L1   0.0001  0.931973
2            L1  0.00001  0.938776
3            L2    0.001  0.891156
4            L2   0.0001  0.904762
5    Upsampling      N/A  0.925170
6  Downsampling      N/A  0.884354

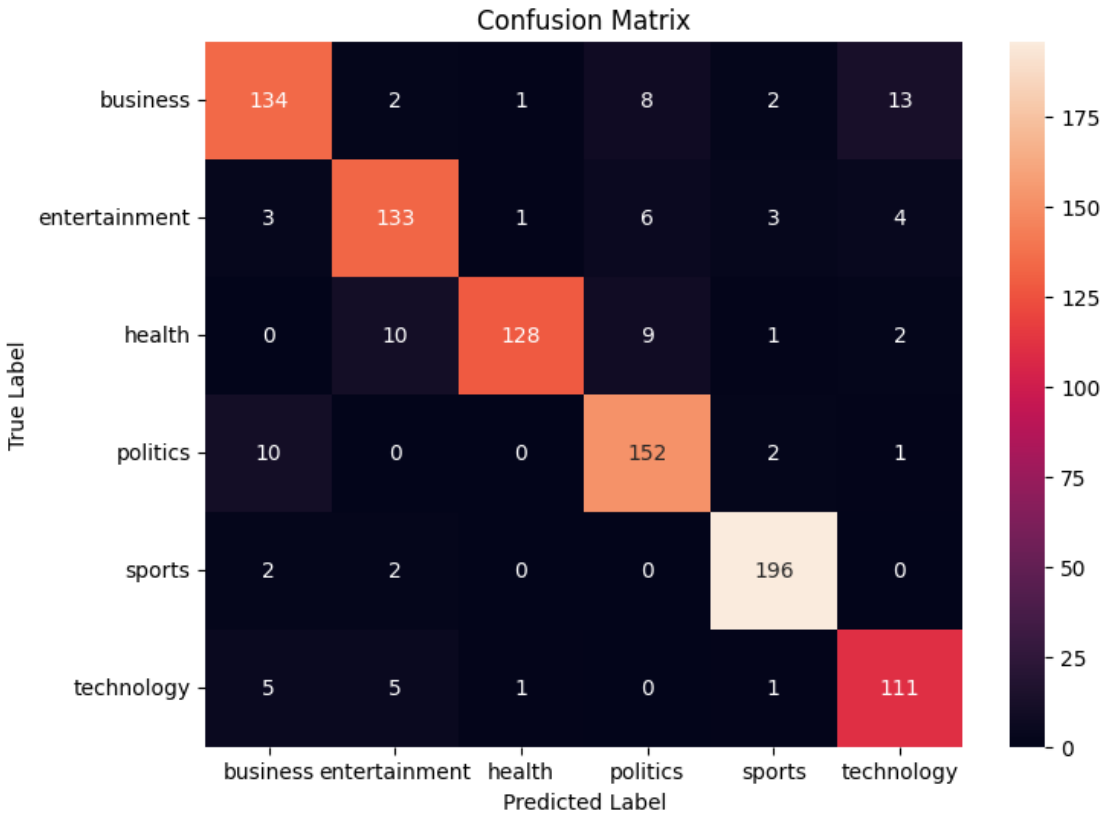
Best technique for isiXhosa: L1 (param: 1e-05) with accuracy 0.9388
```

Evaluation

Final best model evaluations:

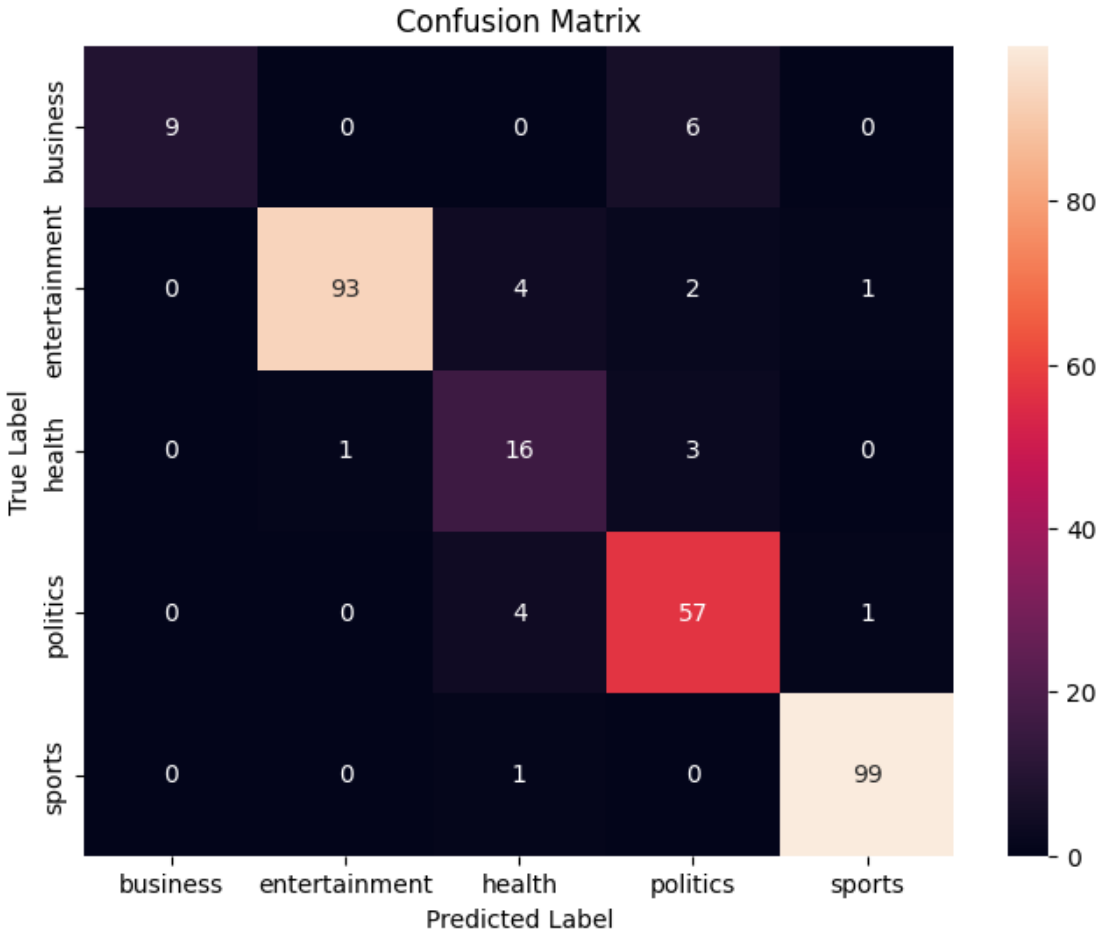
- English:
    - Accuracy: 90.08%
    - Micro-Averaged:
        - Precision: 90.08%
        - Recall: 90.08%
        - F1-Score: 90.08%
    - Macro-Averaged:
        - Precision: 89.9%
        - Recall: 89.69%

- F1-Score: 89.69%



- isiXhosa:
  - Accuracy: 92.26%
  - Micro-Averaged:
    - Precision: 92.26%
    - Recall: 92.26%
    - F1-Score: 92.26%
  - Macro-Averaged:
    - Precision: 88.96%
    - Recall: 84.79%

- F1-Score: 85.64%



Weight Analysis

Top 10 learned weights per class for the **English** model:

```
--- Top features for class: 'business' ---
[('business', '1.355'), ('customers', '1.188'), ('businesses', '1.137'),
('prices', '1.118'), ('energy', '1.078'), ('bank', '1.070'), ('market', '1.051'),
('markets', '0.978'), ('firms', '0.974'), ('rising', '0.955')]

--- Top features for class: 'entertainment' ---
[('singer', '1.256'), ('artist', '1.203'), ('music', '1.077'), ('art', '1.050'),
('stars', '1.047'), ('band', '1.031'), ('artists', '1.009'), ('show', '1.005'),
('fans', '0.970'), ('festival', '0.942')]

--- Top features for class: 'health' ---
[('health', '1.700'), ('patients', '1.444'), ('hospital', '1.399'), ('nhs',
'1.227'), ('covid', '1.145'), ('treatment', '1.139'), ('cases', '1.115'), ('care',
'1.080'), ('nurse', '1.020'), ('hospitals', '1.002')]

--- Top features for class: 'politics' ---
[('government', '1.537'), ('minister', '1.467'), ('sunak', '1.447'), ('rishi',
'1.381'), ('conservative', '1.381'), ('mp', '1.355'), ('prime', '1.240'),
('welsh', '1.191'), ('labour', '1.184'), ('mr', '1.176')]

--- Top features for class: 'sports' ---
```

```
[('sport', '1.672'), ('olympic', '1.455'), ('games', '1.309'), ('win', '1.300'),
('world', '1.280'), ('champion', '1.226'), ('olympics', '1.224'), ('victory',
'1.194'), ('athletes', '1.172'), ('team', '1.169')]

--- Top features for class: 'technology' ---
[('technology', '1.360'), ('tech', '1.292'), ('media', '1.158'), ('users',
'1.136'), ('online', '1.077'), ('use', '1.028'), ('data', '1.015'), ('content',
'1.012'), ('social', '0.990'), ('information', '0.988')]
```

In certain cases, the model has determined very strong associations for certain words. Such as the word `'technology'` being strongly associated with the `'technology'` class - no surprise there. In other cases, it's made a few incorrect associations, such as associating the word `'mr'` with the `politics` class. In this case, it seems that the model is slightly overfitting and not generalizing as well.